Add @, $, and ` to the basic character set

Document #:	P2558R1
Date:	2022-05-11
Project:	Programming Language C++
Audience:	SG16 SG22 EWG

Reply-to:	Steve Downey <sdowney@gmail.com>

Contents

§ 1 Abstract

WG14, the C Standardization committee, is adopting [N2701] for C23.

This will add

Code point	Glyph	Name
U+0024	$	DOLLAR SIGN,
U+0040	@	COMMERCIAL AT
U+0060	`	GRAVE ACCENT

to the basic source character set.

C++ should adopt the same characters for C++26.

§ 2 Motivation

These characters are available in all encoded character sets in common use and everyone assumes that they are available, using them freely in source text. The primary change would be that these characters become available for syntactic purposes. Although using $ in identifiers is a common extension, they were not added to the identifier set in C, and this paper does not propose adding them either. Nor were trigraphs added in C for these characters, and this paper does not propose additional trigraphs or digraphs be added.

The translation model for C makes adding these to their basic source character set, the encoded set for source code before translation, much more compelling. These characters being already in the translation character set as single byte characters makes this less important for C++. Nonetheless, it would be useful to make these available for language purposes as the more conservative C language has agreed there are no functional impediments to their use.

Corentin Jabot discusses the usage in other programming languages extensively in [P2342R0], *For a Few Punctuators More*, q.v.

While it would be possible to add these characters to the grammar of C++ without adding them to the basic character set, that would violate the contract that the basic character set is sufficient for writing C and C++. Digraphs and trigraphs are concessions to ease of keyboarding. It is assumed that the characters represented by those means are available.

§ 3 Implications and Consequences

Because this proposal is not making these characters available for syntactic purposes, the changes are limited to how these characters encoded today, or are represented in source.

§ 3.1 Universal Character Names

Adding these characters to the basic source character set implies that they can no longer be spelled as universal character names outside of literals. An example of such a construct:

```
#include <stdio.h>

#define STR(x) #x

int main()
{
  printf("%s", STR(\u0060)); // U+0060 is ` GRAVE ACCENT
}
```

This is currently rejected by GCC ‘error: universal character is not valid in an identifier’, although this seems to be a bug, and the code is accepted by clang and msvc.

§ 3.2 Literal Encoding

Adding these characters to the basic character set means these will have to be encoded in a single byte, with positive value when used as a `char`. This is true for all POSIX encoded character sets, as @, $, and ` are part of the portable character set. This also implies they are available in all POSIX locales, and in particular the “POSIX” locale, which is equivalent to the “C” locale. [POSIX] See 6. Character Set

§ 3.3 Runtime Encoding

A locale that does not provide for these characters would be non-conforming. Interpreting the literal encoding in any encoded character set, including the “C” LC_CTYPE character set if it does not match the literal encoding, is already at best unspecified. Substitution ciphers are apparently conforming, although misleading. There is a long history of interpreting the Yen sign, ¥, as a path separator on Windows exactly because of these encoding aliasing issues.

POSIX, which has the ultimate definition of locales and how they work, defines the Portable Character Set that are required to be encoded. They are not invariant, and may appear in different locations in different encodings.

IBM defines a portable EBCDIC set to conform with the X/Open portable character set. However, it is documented to contain Accent Acute, which is not part of the portable set defined by X/Open or POSIX. Examining EBCDIC code pages, I did not find any that did not encode U0060. $ and @ are included in the EBCDIC portable character set.

§ 3.4 Source Encoding and Representation

There is a rule that characters in the basic character set may not be expressed as UCNs, unless inside a character or string literal. For C there are issues for characters in comments. This is not the case for C++. In non-comment contexts, these characters are currently not allowed in portable source outside of strings and character literals, so the spelling of the character is irrelevant, so other than the stringification example above, this seems unlikely to break real world code.

For extensions that allow, for example, $ in identifiers, no one outside of compiler test suites, is likely to use a UCN to spell that.

C++ places no constraints on source encoding. The closest we have is the in-flight requirement that implementations that accept files be required to accept UTF-8, and UTF-8 encodes these characters.

§ 3.5 Raw String Literals

The new characters would be allowed in the raw delimiters, as there seems no reason to exclude them. The current list of exclusions is because white space, parentheses, and back slash could cause parsing ambiguity in the paired delimiter strings.

§ 3.6 Header names

The grammar productions for header names uses the translation character set. It is conditionally supported with implementation defined semantics if `\\` is allowed, from which we can infer that universal character names are conditionally supported. If anyone was using UCNs to represent the new characters in a header, implementations could continue to interpret them, despite the rule of UCNs not being a valid representation of characters in the basic character set.

Footnote 14 from [lex.header]

> Thus, a sequence of characters that resembles an escape sequence can result in an error, be interpreted as the character corresponding to the escape sequence, or have a completely different meaning, depending on the implementation.

§ 4 Wording

These changes are relative to [N4901] “Working Draft, Standard for Programming Language C++”

Modify [lex.charset] as follows:

2 The basic character set is a subset of the translation character set, consisting of ~~96~~99 characters as specified in Table 1.

Modify [tab:lex.charset.basic] with the following additions:

```
U+0009                CHARACTER TABULATION
U+000B                LINE TABULATION
U+000C                FORM FEED
U+0020                SPACE
U+000A                LINE FEED                    new-line
U+0021                EXCLAMATION MARK             !
U+0022                QUOTATION MARK               "
U+0023                NUMBER SIGN                  #

U+0024                DOLLAR SIGN                  $

U+0025                PERCENT SIGN                 %
U+0026                AMPERSAND                    &
U+0027                APOSTROPHE                   '
U+0028                LEFT PARENTHESIS             (
U+0029                RIGHT PARENTHESIS            )
U+002A                ASTERISK                     *
U+002B                PLUS SIGN                    +
```

```
U+002C                COMMA                          ,
U+002D                HYPHEN-MINUS                   -
U+002E                FULL STOP                      .
U+002F                 SOLIDUS                       /
U+0030 .. U+0039      DIGIT ZERO .. NINE             0 1 2 3 4 5 6 7 8 9
U+003A                COLON  :
U+003B                SEMICOLON                      ;
U+003C                LESS-THAN SIGN                 <
U+003D                EQUALS SIGN                    =
U+003E                GREATER-THAN SIGN              >
U+003F                QUESTION MARK                  ?

U+0040                COMMERCIAL AT                  @

U+0041 .. U+005A      LATIN CAPITAL LETTER A .. Z  A B C D E F G H I J K L
        M
                                                    N O P Q R S T U V W X Y
        Z
U+005B                LEFT SQUARE BRACKET            [
U+005C                REVERSE SOLIDUS                \
U+005D                RIGHT SQUARE BRACKET           ]
U+005E                CIRCUMFLEX ACCENT              ^
U+005F                LOW LINE                       _

U+0060                GRAVE ACCENT                   `

U+0061 .. U+007A      LATIN SMALL LETTER A .. Z    a b c d e f g h i j k l
        m
                                                    n o p q r s t u v w x y
        z
U+007B                LEFT CURLY BRACKET             {
U+007C                VERTICAL LINE                  |
U+007D                RIGHT CURLY BRACKET            }
U+007E                TILDE                          ~
```

Add to Annex C

C.n C++ and ISO C++ 2023 [diff.cpp23]

C.n.1 [lex]: lexical conventions [diff.cpp23.lex]

1 **Affected subclause:** [lex.charset]

Change: The characters $, @, and ` may not be represented as a *universal-character-name* outside a literal.

Rationale: Inclusion of these characters in the basic character set.

Effect on original feature: The characters were not allowed in semantically meaningful text outside of literals, but could still be present in early phases of translation as *universal-character-names*.

[Example 1:

```
#include <stdio.h>

#define STR(x) #x

int main()
{
  printf("%s", STR(\u0024)); // UCN for $ was allowed, now it is not
}
```

— end example]

[Example 2:

```
#include <stdio.h>

#define EAT(x)

int main()
{
  EAT(\u0024) // UCN for $ was allowed, now it is not
}
```

— end example]

5 References

[N2701] Philipp Klaus Krause. @ and $ in source and execution character set.
http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2701.htm

[N4901] Thomas Köppe. 2021-10-22. Working Draft, Standard for Programming Language C++.
https://wg21.link/n4901

[P2342R0] Corentin Jabot. 2021-03-25. For a Few Punctuators More.
https://wg21.link/p2342r0

[POSIX] IEEE and The Open Group. The Open Group Base Specifications Issue 7, 2018 edition.
https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/contents.html